Лабораторно упражнение 10

Поведенчески шаблони. Шаблон „Стратегия“ (Strategy)

I. Теоретична част

1. Шаблон „Strategy“ - същност

Шаблон „Стратегия“ принадлежи към групата на поведенческите шаблони. Той дефинира фамилия (множество) алгоритми, капсулира всеки един от тях чрез разделяне в отделни класове и ги прави взаимозаменяеми. Този шаблон позволява на изменение на алгоритъма нзависимо от клиента, който го използва.

Цел и мотивация

Цел:

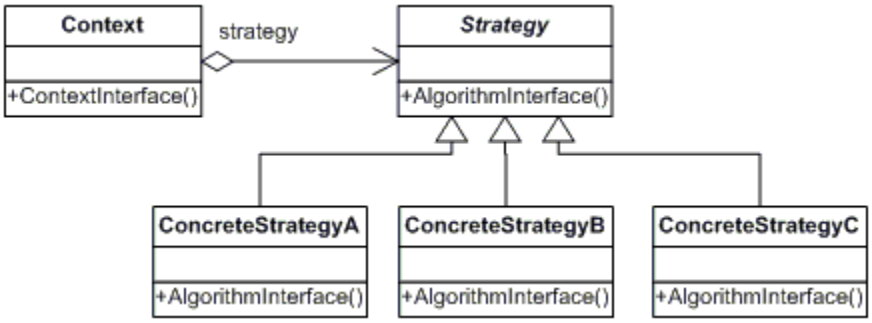
Шаблон „Стратегия“ определя семейство алгоритми, капсулира всеки един от тях и ги прави зваимозаменяеми. Стратегията позволява да се промени алгоритъма независимо от клиента, който го използва.

Мотивация:

Шаблон „Стратегия“ се явява един от най-често използваните. На практика когато се използва наследяване, всъщност се прилага стратегия. Всеки пък, когато програмистът се абстрахира от определен процес, поведение или алгоритъм на базовия клас или интерфейс се използва стратегия. „Стратегията“ е фундаментален шаблон, тъй като се проявява в повечето други класически шаблониза дизайн. Например, „Метод Фабрика“ е стратегия за създаване на един обект; „Абстрактна Фабрика“ е стратегия за създаване на семейство обекти; „Строител“ е стратегия за построяване на обект.

1.1. UML диаграма

На фиг. 1 е представена примерна UML диаграма на класовете, участващи в шаблона „Стратегия“



Фиг. 1. UML диаграма на шаблон „Стратегия“

В диаграмата участват следните класове и обекти:

- Strategy – декларира общ интерфейс за всички алгоритми, съответстващи на всички „стратегии“. Клас Context използва този интерфейс, за да извика алгоритъма, съответстващ на конкретната стратегия.
- ConcreteStrategy – имплементира конкретна стратегия (ConcreteStrategyA, ConcreteStrategyB, ConcreteStrategyC) т.е. конкретния алгоритъм

- Context – конфигурира се от обект ConcreteStrategy, управлява този обект и може да дефинира интерфейс, който позволява на стратегията да достъпва данните.

В диаграмата може да участва и клиентски клас.

1.2. Програмна реализация

```
using System;
namespace Strategy
{
    class Program
    {
        static void Main(string[] args)
        {
            Context context;
            // Последователно използване на 3 стратегии
            context = new Context(new ConcreteStrategyA());
            context.ContextInterface();
            context = new Context(new ConcreteStrategyB());
            context.ContextInterface();
            context = new Context(new ConcreteStrategyC());
            context.ContextInterface();

        }
    }
    // клас, съответстващ на инерфейса стратегия
    public abstract class Strategy
    {
        public abstract void AlgorithmInterface();
    }
    // клас, съответстващ на конкретна стратегия A
    public class ConcreteStrategyA : Strategy
    {
        public override void AlgorithmInterface()
        {
            Console.WriteLine("Called ConcreteStrategyA.AlgorithmInterface()");
        }
    }
    // клас, съответстващ на конкретна стратегия B
    public class ConcreteStrategyB : Strategy
    {
        public override void AlgorithmInterface()
        {
            Console.WriteLine(
                "Called ConcreteStrategyB.AlgorithmInterface()");
        }
    }
    // клас, съответстващ на конкретна стратегия C
    public class ConcreteStrategyC : Strategy
    {
        public override void AlgorithmInterface()
        {
            Console.WriteLine(
                "Called ConcreteStrategyC.AlgorithmInterface()");
        }
    }
    public class Context
    {
        Strategy strategy;
        // конструктор
        public Context(Strategy strategy)
        {
            this.strategy = strategy;
        }
        public void ContextInterface()
```

```
        {
            strategy.AlgorithmInterface();
        }
    }
}
```

Представената програмна реализация е на структурна за шаблон „Стратегия“

2. Пример за реализация на шаблон „Стратегия“

Пример за реализация на шаблон „Стратегия“ създаване на градина, в която цветята са подбрани по определена цветова гама. Например градина, в която са подбрани цветя цъфтящи в жълто или градина, в която цветята са в розово.

Абстрактен клас `Garden` съответства на интерфейса „Стратегия“.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Strategy
{
    public abstract class Garden
    {

        protected List<string> flowers = new List <string>();
        public abstract void CreateGarden();
        public void ShowGarden()
        {
             foreach(string flower in flowers)
             {
               Console.WriteLine(flower);
             }
        }
    }
}
```

Клас `Garden` се наследява от класове `VioletGarden`, `YellowGarden` и `PinkGarden`.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Strategy
{
    class VioletGarden:Garden
    {
        public override void CreateGarden()
        {
            flowers.Add("Трителея Koninging fabiola ");
            flowers.Add("Гладиол Cote D'azur");
            flowers.Add("Фрезия двойна синя");
            flowers.Add("Гладиол Callianthus ");
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Strategy
```

```
{
    class YellowGarden:Garden
    {
        public override void CreateGarden()
        {
            flowers.Add("Иксия mix ");
            flowers.Add("Гладиол жълт");
            flowers.Add("Фрезия");
            flowers.Add("Крокосмия");
        }
    }
}
using System;
using System.Collections.Generic;
using System.Text;

namespace Strategy
{
    class PinkGarden:Garden
    {
        public override void CreateGarden()
        {
            flowers.Add("Оксалис");
            flowers.Add("Гладиол розов");
            flowers.Add("Фрезия двойна розова");
            flowers.Add("Гладиол Callianthus");
        }

    }
}
```

Клас `Context` има следния код:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Strategy
{
    class Context
    {
             Garden myGarden;
            public Context(Garden garden)
            {
                this.myGarden = garden;
            }
            public void ContextInterface()
            {
                myGarden.CreateGarden();
                myGarden.ShowGarden();
            }
    }
}
```

Клиентския клас има следния код:

```
using System;
namespace Strategy
{
    class Program
    {
```

```
        static void Main(string[] args)
        {
            Context context;

            string color;
            Console.Write("Color:");
            color = Console.ReadLine();
            switch(color.ToLower())
            {
                case "pink":
                    context = new Context(new PinkGarden());
                    context.ContextInterface();
                    break;
                case "yellow":
                    context = new Context(new YellowGarden());
                    context.ContextInterface();
                    break;
                case "violet":
                    context = new Context(new VioletGarden());
                    context.ContextInterface();
                    break;
                default:
                    Console.WriteLine("There no such garden");
                    break;

            }
        }
    }
}
```

II. Задачи за изпълнение

1. Да се създаде приложение, което използвайки шаблон „Стратегия“ сортира масив от цели числа като използват 3 метода за сортиране. Да не се използва метод Sort на клас Array! Да се изберат 3 алгоритъма от познатите методи за сортиране, например метод на прякото вмъкване, метод на пряката селекция, метод на мехурчето, бързо сортиране и др.

Допълнително условие: Да се пресмятне колко размени прави всеки един от методите при сортиране на даден масив и да се направи сравнение между отделните алгоритми.